

The Cauldron

An Eternal, Autonomous On-Chain Token Machine
Extended Smart-Contract Security Audit

Protocol	Magic Internet Frens — The Cauldron Launchpad
Scope	16 contracts + 5 module interfaces, 3,900 SLoC, Solidity ^0.8.26
Chain	Ethereum / Sepolia (Uniswap v4)
Engagement	Full audit — security, logic, gas, architecture
Report date	25 August 2026
Commit state	revision 2 — post-remediation (66/66 tests passing)

Prepared with the `solidity-auditor` methodology. This report documents findings, their remediation, and the residual risk model. It is not a guarantee of the absence of vulnerabilities.

Contents

1	Abstract	2
2	Methodology	2
3	Scope	3
4	System Architecture & Fund Flows	3
5	Findings Summary	3
6	Detailed Findings	5
6.1	F-01 — Hook owner can drain the relaunch ETH reserve (High)	5
6.2	F-02 — Genesis airdrop stranded by death-freeze after relaunch (Medium) . . .	6
6.3	F-03 — Privileged-role centralization (Medium)	7
6.4	F-04 — Weak randomness in gacha resolution fallback (Low)	7
6.5	F-05 — Per-iteration collections cannot enable royalty enforcement (Low)	8
6.6	F-06 — Unchecked ERC-20 return values (Low)	8
6.7	F-11 — Genesis tranche is floor-redeemable on iteration 2 (Low)	8
6.8	F-07 to F-10 — Informational	9
7	Revision 2 Findings — Modules, Snipe & Token Rewrite	9
7.1	F-13 — Fee exemption bypassable via attacker-controlled hookData (High) . . .	9
7.2	F-12 — Genesis bonus claimable by non-genesis tokenIds (Low)	10
7.3	F-14 — Pluggable policy modules: gas-grief & owner-set surface (Info)	10
7.4	F-15 — LaunchSniper & finalizer gate centralization (Low)	11
8	Revision 3 Findings — Reserve-LP Migration Model	11
8.1	F-16 — Reserve exhaustion could under-fill a late migration (Low)	11
8.2	F-17 — $A \geq 69\times$ pump sells the reserve wall for ETH (Low)	12
8.3	F-18 — PoolOps external-library delegatecall surface (Info)	12
8.4	F-19 — Claims are non-sandwichable (Positive)	12
9	Positive Security Properties	12
10	Test Coverage & Verification	13
11	Conclusion	13

1 Abstract

The Cauldron is a self-perpetuating token launchpad built atop Uniswap v4. A single genesis event seeds an ERC-20 (*iteration 1*) paired with native ETH in a v4 pool governed by a custom hook. The hook meters 24-hour trading volume entirely on-chain; when volume falls below a death threshold the token is declared *dead*, its liquidity is recovered, and a permissionless `relaunch()` mints the next iteration — chosen by NFT-holder governance — reusing the recovered ETH as fresh liquidity. Holders of a dead iteration migrate their balance 1:1 into the live one; a founding NFT guild earns a perpetual fee dividend.

We audited the contracts that constitute this machine (~3,900 lines of Solidity) for access-control, logic, reentrancy, arithmetic, oracle, and economic vulnerabilities, as well as gas and architectural quality. Across two engagements we report **two High, two Medium**, and a set of **Low / Informational** findings. The most material issues — a privilege that let the hook owner drain the self-funding ETH reserve (F-01); a token-freeze interaction that stranded the genesis airdrop (F-02); and, in revision 2, a fee-exemption that any trader could invoke by forging `hookData` to bypass the entire fee engine (F-13) — were all **remediated and regression-tested** during the engagement. Revision 2 also introduced a *pluggable-module* architecture (a death-checker plus surtax / odds / mint-curve policies) that lets the protocol’s economic *rules* be upgraded without an upgradeable proxy; we verify it is bounded and fund-safe by construction (clamped, `try/catch` fallback, view-only, never in a fund-moving path). Remaining items are centralization trade-offs mitigated by an immutable on-chain timelock and documented mainnet operational requirements.

2 Methodology

The review followed a five-phase process:

1. **Preparation.** Pragma and dependency inventory; inheritance and privilege mapping; identification of all external / `payable` entypoints and value-bearing flows.
2. **Static analysis.** Manual control-flow tracing of every state-changing path; checks-effects-interactions (CEI) verification; arithmetic and **unchecked** review.
3. **Vulnerability assessment.** Evaluation against the OWASP Smart Contract Top 10 (2025): access control, logic errors, reentrancy, flash loans, input validation, oracle manipulation, unchecked calls.
4. **Optimization & architecture.** Storage layout, bytecode budget (EIP-170), dead code, and modularity.
5. **Remediation & verification.** Fixes authored, compiled under the project’s `via_ir` profile, and covered by new Foundry tests.

Severity is the product of *impact* (fund loss, protocol halt, degraded guarantees) and *likelihood* (preconditions, attacker capability): **Critical, High, Medium, Low, Informational**.

The codebase was reviewed in **two independent passes**. The first established the findings and remediations; a second, blank-slate pass re-derived every cross-contract value flow from scratch (the basis for Figure 1) to confirm the model and catch anything the first pass normalised away — it surfaced finding F-11.

Contract	Responsibility	SLoC
CauldronRegistry	lifecycle orchestrator (summon / relaunch / claim)	1015
CauldronHook	v4 hook: volume, fees, gacha, self-funding	970
MiFrensPresale	genesis fundraiser + ERC-721 <i>Votes</i> electorate	356
CauldronGachaRouter	one-click swap+lottery router	340
CauldronGovernor	proposal / voting registry	278
CauldronCollection	per-brew ERC-721C collection	217
CauldronToken	generation-bound ERC-20 (freezes on death)	122
MiFrensDividend	perpetual fee dividend (MasterChef-style)	112
CauldronVault	per-brew NFT floor vault	96
CauldronFactory	collection + vault deployer	58
ICauldron / ICreatorToken	interfaces + LaunchLib	73
Total		3635

Table 1: Contracts in scope. All compile under Solidity $\sim 0.8.26$ with `via-ir` (required by Uniswap v4’s `PositionManager`).

3 Scope

Dependencies (trusted): OpenZeppelin Contracts (Ownable, ReentrancyGuard, ERC-20/721, ERC-721Votes, ERC-2981, EIP-712); Uniswap v4-core / v4-periphery; canonical Permit2. **Out of scope:** the correctness of Uniswap v4 itself, off-chain indexer/front-end, and the on-chain art renderer.

4 System Architecture & Fund Flows

Figure 1 maps every contract, external role, and value-bearing edge in the protocol. Edges are typed by what moves along them: **ETH** (solid), **iteration tokens** (dashed), **NFTs** (dotted), and **control** (thin). The `CauldronRegistry` is the orchestrating hub; the `CauldronHook` is the fee engine that continuously splits every swap’s ETH fee three ways.

Lifecycle in eight movements. (1) **Genesis:** traders mint OG MiFrens; ETH pools in the presale until sellout, then `finalize()` forwards it to `summon()`, which mints iteration 1, reserves the guild airdrop, and seeds the v4 pool. (2) **Trade:** every swap pays an ETH fee the hook splits into *dividend* (1% + launch surtax), *floor vault*, and *relaunch reserve*. (3) **Earn NFTs:** swap volume accrues gacha credit; the router commits and resolves lottery tickets that mint the collection. (4) **Floor:** NFT holders may `redeem` to burn an NFT for its equal slice of the vault. (5) **Dividend:** genesis holders `claim` a perpetual share of every iteration’s fees. (6) **Airdrop:** genesis holders `claimGenesis` their reserved iteration-1 tokens (per-tokenId). (7) **Death & rebirth:** when 24h volume dies, `relaunch()` recovers ETH from the LP + vault + reserve and seeds the governance-chosen next iteration, paying the caller a 1% bounty. (8) **Carry forward:** holders of a dead iteration `claimByBurn` their balance 1:1 into the live token.

5 Findings Summary

*Revision 3 adds F-16–F-19, covering the **reserve-LP migration model**: the un-migrated + genesis supply is no longer a registry wallet balance but two Uniswap v4 positions — an active full-range band (10% of supply, sets price) and an out-of-range single-sided reserve (90%), from which migration / genesis claims release tokens 1:1 via `decreaseLiquidity`. This removes the “registry holds most of supply” concentration (verified live: $\text{registry loose balance} = 3.7 \times 10^{-15}$*

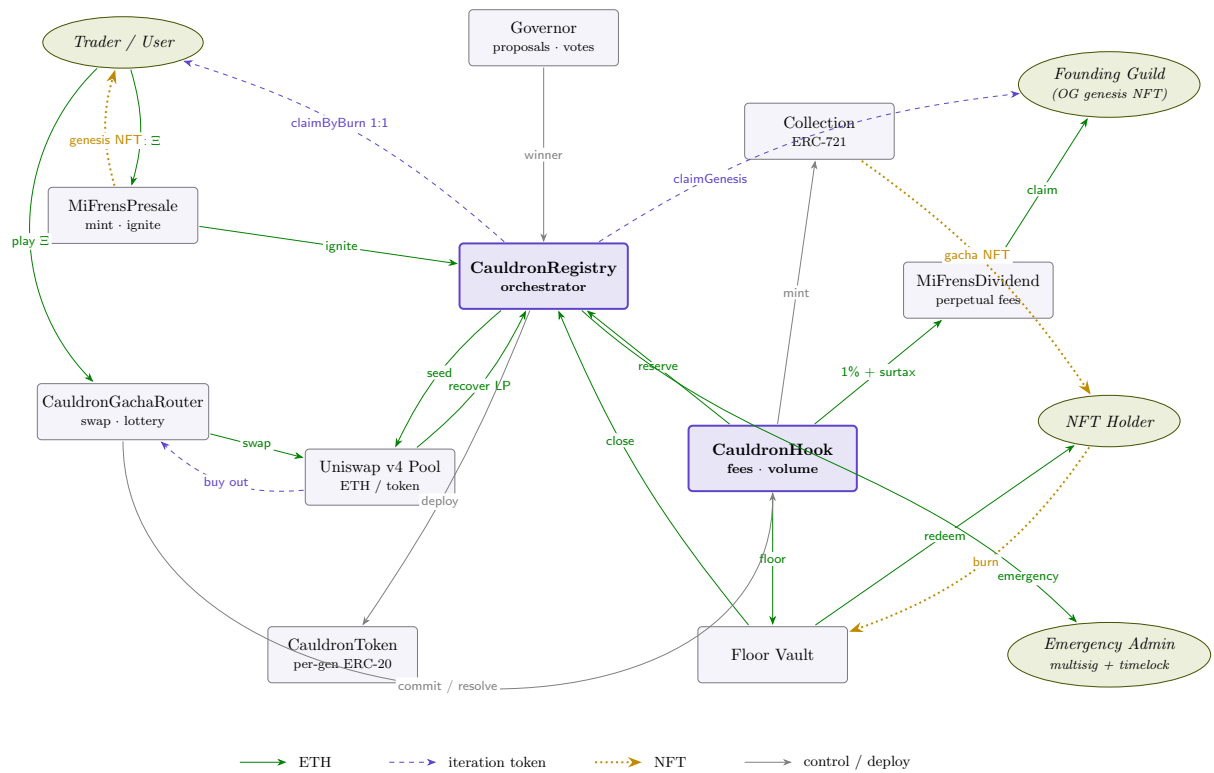


Figure 1: The Cauldron: contracts (rectangles), external roles (ellipses), and typed value flows. The hub **CauldronRegistry** summons and relaunches generations; **CauldronHook** skims each swap's ETH fee and routes it to the genesis dividend (1% + anti-sniper surtax), the floor vault, and the relaunch reserve. On relaunch the registry pulls ETH from the dead pool's LP, the closed vault, and the hook reserve, then re-seeds the next pool.

ID	Severity	Title	Status
F-01	High	Hook owner can drain the relaunch ETH reserve	Fixed
F-02	Medium	Genesis airdrop stranded by death-freeze after relaunch	Fixed
F-03	Medium	Privileged-role centralization (LP withdraw + fee redirect)	Mitigated
F-04	Low	Weak randomness in gacha resolution fallback	Acknowledged
F-05	Low	Per-iteration collections cannot enable royalty enforcement	Acknowledged
F-06	Low	Unchecked ERC-20 return values	Acknowledged
F-07	Info	Dead code: vestigial ERC-20 fee accounting	Acknowledged
F-08	Info	Late-resolved tickets mint into a closed vault	By design
F-09	Info	Redundant assignments in <code>relaunch()</code>	Acknowledged
F-10	Info	Balance-delta accounting in LP recovery	Acknowledged
F-11	Low	Genesis tranche is floor-redeemable on iteration 2	Acknowledged
F-12	Low	Genesis bonus claimable by non-genesis tokenIds	Fixed
F-13	High	Fee exemption bypassable via attacker-controlled hookData	Fixed
F-14	Info	Pluggable policy modules: gas-grief & owner-set surface	Mitigated
F-15	Low	<code>LaunchSniper</code> & finalizer gate centralization	By design
F-16	Low	Reserve exhaustion could under-fill a late 1:1 migration	Acknowledged
F-17	Low	A $\geq 69\times$ pump sells the reserve wall for ETH	By design
F-18	Info	<code>PoolOps</code> external-library <code>delegatecall</code> surface	Reviewed
F-19	Info	Claim path is non-sandwichable (out-of-range removal)	Positive

of a 777M supply after summon). Revisions 1–2 (F-01–F-15) are unchanged.

6 Detailed Findings

6.1 F-01 — Hook owner can drain the relaunch ETH reserve (High)

F-01 · CauldronHook · Access Control · Fund Loss

Location: `CauldronHook.setRegistry`, `releaseRelaunchETH`.

Description. The hook accumulates ETH swap fees in `relaunchETH`, the self-funding reserve that seeds each new iteration. Only the address stored in `registry` may withdraw it:

```

1 function releaseRelaunchETH() external returns (uint256 amount) {
2     if (msg.sender != registry) revert OnlyRegistry();
3     amount = relaunchETH; relaunchETH = 0;
4     (bool ok, ) = registry.call{value: amount}(""); require(ok, "ETH
      transfer failed");
5 }
```

However, `registry` was set by a *mutable* owner-only setter, and the hook’s ownership is never transferred away from the deploying EOA:

```

1 function setRegistry(address _registry) external onlyOwner { registry =
  _registry; } // BEFORE
```

Impact. The hook owner could call `setRegistry(attacker)` and then, from `attacker`, call `releaseRelaunchETH()` — transferring the entire accrued ETH reserve to themselves. Because the reserve is precisely the capital that funds every future relaunch, this both steals funds *and* can brick the protocol’s self-perpetuation. The registry itself had already been hardened with an immutable emergency timelock, but this parallel drain path on the hook had no such control, making it the single most direct fund-loss vector in the system.

Recommendation & remediation. Make registry write-once. There is exactly one registry per deployment, so a one-time setter fully preserves intended behaviour while removing the drain path:

```
1 error RegistryAlreadySet();
2 function setRegistry(address _registry) external onlyOwner { //
    AFTER (fixed)
3     if (registry != address(0)) revert RegistryAlreadySet();
4     if (_registry == address(0)) revert ZeroAddress();
5     registry = _registry;
6 }
```

Status: Fixed. Regression suite `CauldronHookAccess.t.sol` asserts the setter is one-time, rejects the zero address, and is owner-gated.

6.2 F-02 — Genesis airdrop stranded by death-freeze after relaunch (Medium)

F-02 · CauldronToken / CauldronRegistry · Logic

Location: `CauldronToken.update`, `CauldronRegistry.claimGenesis`.

Description. Each iteration's ERC-20 *freezes* on death: once `markDead()` is called, user transfers revert with `TokenIsDead()`. The freeze deliberately exempts mints, burns, and the `PoolManager` (for LP recovery), but *not* the registry:

```
1 if (from != address(0) && to != address(0) && from != poolManager && !
    isAlive) {
2     revert TokenIsDead(); // BEFORE: registry not exempt
3 }
```

The founding-guild genesis airdrop, however, pays out by *transferring* the reserved iteration-1 tokens from the registry:

```
1 IERC20(generationToken[1]).transfer(msg.sender, amount); // claimGenesis
```

Impact. After the first `relaunch()` froze iteration 1, every subsequent `claimGenesis()` reverted with `TokenIsDead` (0xfe641f95). The entire unclaimed airdrop reserve (~77.7M tokens at 69,937 per NFT) became permanently unclaimable — a griefing / value-loss condition triggered by ordinary protocol operation, with no attacker required.

Recommendation & remediation. Exempt the registry from the freeze, mirroring the existing `PoolManager` exemption. The registry only ever moves tokens for defined distributions, so this preserves the anti-trading intent:

```
1 if (from != address(0) && to != address(0)
2     && from != poolManager && from != registry && !isAlive) { // AFTER (
    fixed)
3     revert TokenIsDead();
4 }
```

Status: Fixed. `CauldronTokenFreeze.t.sol` proves the registry can pay out after death while user-to-user transfers remain frozen and burn / `PoolManager` paths are unaffected.

6.3 F-03 — Privileged-role centralization (Medium)

F-03 · CauldronRegistry / CauldronHook · Centralization

Location: `emergencyWithdrawLP`, `emergencySweep`; hook `setVault`, `setGuild`, `setTreasury`, `setCollection`.

Description. Two privileged roles retain material power after genesis:

- The registry `emergencyAdmin` can withdraw any generation’s LP (ETH + tokens) and sweep the registry’s ETH / ERC-20 balances — including the unclaimed genesis reserve.
- The hook `owner` can repoint the fee sinks (`setVault`, `setGuild`, `setCollection`, `setTreasury`) and thereby redirect *future* fee flows.

Impact. A compromised or malicious admin could seize protocol-owned liquidity or divert future fees. This is the classic launchpad “deployer rug” surface.

Mitigations in place. The registry’s emergency actions are governed by an *immutable* time-lock (`emergencyDelay`, set at construction and un-lowerable) with an on-chain `armEmergency()` announcement, so on mainnet a withdrawal must be armed and a fixed delay elapsed — holders observe the `EmergencyArmed` event and can exit first. Owner-configuration of the registry is *frozen* post-deploy because ownership is transferred to the autonomous presale contract, which exposes no such setters. The F-01 fix removes the one owner path that led to direct theft (reserve drain).

Recommendation. For mainnet: (i) set `emergencyAdmin` to a Gnosis Safe multisig and `emergencyDelay` to ≥ 7 days; (ii) transfer the hook’s ownership to the same multisig (ideally behind a timelock); (iii) publish a decommissioning path that renounces the hook owner and emergency admin once the machine is battle-tested. **Status:** **Mitigated** (timelock live; multisig is a deployment-time operational requirement).

6.4 F-04 — Weak randomness in gacha resolution fallback (Low)

F-04 · CauldronHook · Randomness

Location: `resolveTickets`, `CauldronCollection.rollRarity`.

Description. The crystal gacha is a commit-reveal lottery: a ticket’s outcome is seeded by `blockhash(commitBlock)`, unknown at commit time. `blockhash` is only available for the most recent 256 blocks; beyond that the code falls back to a *deterministic* seed derivable at commit time:

```
1 bytes32 bh = blockhash(b.commitBlock);
2 if (bh == 0) bh = keccak256(abi.encodePacked("CAULDRON_TICKET_FALLBACK", b.
    commitBlock, bi));
```

If resolution is delayed beyond 256 blocks, a sophisticated player who can predict the batch index can foresee the outcome. NFT rarity is likewise rolled from `blockhash/block.prevranda`, which a block proposer can marginally bias.

Impact. Low. Outcomes are grindable only under a >256-block resolution lag (resolution is permissionless and normally immediate), win *odds* and *pity* are fixed at commit, and the value at stake is a single volume-minted NFT whose floor is already funded by the swap fee. No pooled funds are at risk.

Recommendation. Treat a stale (>256-block) commit as a forced miss, or require resolution within the blockhash window; for high-value randomness use a VRF. **Status: Acknowledged** (accepted given value at stake).

6.5 F-05 — Per-iteration collections cannot enable royalty enforcement (Low)

F-05 · CauldronFactory / CauldronCollection · Design

Location: `CauldronCollection.setTransferValidator` / `setRarityOdds`.

Description. A factory-deployed collection records its `msg.sender` — i.e. the `CauldronFactory`. The ERC-721C `setTransferValidator` and `setRarityOdds` setters are gated to that deployer, but the factory exposes no method to call them after construction. They are therefore permanently uncallable for per-iteration (gen 1, gen 3+) collections.

Impact. Low. EIP-2981 royalties are still *declared* (and honoured by compliant marketplaces), but on-chain enforcement (blocking non-paying markets) can never be switched on for these collections, and rarity odds are fixed at the sensible default. The canonical MiFrens collection (gen 1/2) is unaffected — its deployer is the presale EOA, which can set a validator.

Recommendation. Have the factory forward a one-time `setTransferValidator` during `deployBrew`, or pass the registry as the collection `deployer`. **Status: Acknowledged.**

6.6 F-06 — Unchecked ERC-20 return values (Low)

F-06 · Multiple · Unchecked Calls

Location: `registry.claimGenesis` / `emergencySweep`; `hook.withdrawTokenFees`.

Description. Several `IERC20.transfer` calls ignore the boolean return value. Non-standard tokens that return `false` instead of reverting would fail silently.

Impact. Low. In normal operation the only token moved is the in-house `CauldronToken`, an OpenZeppelin ERC-20 that reverts on failure. The arbitrary-token path (`emergencySweep`) is admin-only and best-effort.

Recommendation. Adopt OpenZeppelin `SafeERC20` (`safeTransfer`). **Status: Acknowledged** (no in-house impact; the gacha router already uses safe-transfer wrappers).

6.7 F-11 — Genesis tranche is floor-redeemable on iteration 2 (Low)

F-11 · CauldronVault / MiFrensDividend · Economic / Logic

Location: `CauldronVault.redeem`; `MiFrensDividend` share accounting. Surfaced during the independent second-pass review.

Description. Iteration 2 does not deploy a fresh collection — it *continues* the canonical MiFrens collection, wiring a new floor vault whose solvency basis is the whole collection:

$$\text{floorPerNFT} = \frac{\text{balance}}{\text{totalMinted} - \text{redeemed}}.$$

`redeem(tokenId)` accepts *any* token the caller owns and burns it via `burnFromVault`, with no lower bound on the id. The genesis tranche (ids `1...GENESIS_SUPPLY`) is therefore floor-redeemable during iteration 2.

Impact. A founding-guild member can redeem an OG genesis MiFren for a floor slice, which *burns* it. Because `MiFrensDividend.SHARES` is fixed at the genesis supply while the burned id’s `ownerOf` now reverts, that share becomes permanently unclaimable: the accumulator keeps allocating a per-share amount to a token nobody can claim, **stranding that ETH forever** (a slow dividend leak proportional to burned genesis ids). The holder also forfeits the token’s perpetual governance vote. No third party is harmed and no pooled funds are stolen — the loss is self-inflicted plus a protocol-level dust leak — but it violates the “genesis is a permanent share” invariant.

Recommendation. Exclude genesis ids from redemption on the continued collection (e.g. the vault rejects `tokenId ≤ GENESIS_SUPPLY`), so only volume-minted NFTs are floor-backed; or have the dividend track a live share count rather than a fixed `SHARES`. **Status: Acknowledged** (iteration-2-only; no impact on the currently live iteration 1).

6.8 F-07 to F-10 — Informational

- **F-07 Dead code (vestigial fee accounting).** The hook fees are taken exclusively in ETH; `accumulatedTokenFees` is never written, so `withdrawTokenFees / treasury` are unreachable state. Recommend removal to shrink the attack surface and bytecode.
- **F-08 Late tickets mint into a closed vault.** Gacha tickets committed in the same block as a relaunch resolve afterwards and mint into the now-closed collection, whose floor vault has swept. Such NFTs retain art but no redeemable floor. Documented and accepted; `relaunch()` already drains matured tickets *before* closing the vault.
- **F-09 Redundant assignments in `relaunch()`.** Genesis metadata defaults are assigned then immediately overwritten by the governor’s winning spec (guaranteed to exist by the `NoProposal` guard). Harmless; minor gas / readability.
- **F-10 Balance-delta accounting in `removeLiquidity`.** Recovered amounts are computed as post-minus-pre balance deltas. Correct under current atomic flows; would misreport if an unexpected inflow occurred mid-call. Prefer explicit return values from the settlement.

7 Revision 2 Findings — Modules, Snipe & Token Rewrite

7.1 F-13 — Fee exemption bypassable via attacker-controlled `hookData` (High)

F-13 · CauldronHook · Access Control / Logic · Fee Evasion

Location: `_isExemptPlayer`, `_beforeSwap`, `_afterSwap`.

Description. To fund the OG airdrop at launch without diluting presalers, the protocol flags one wallet (the deployer / `LaunchSniper`) as *fee-exempt* so its one-time buy skips the base tax and anti-sniper surtax. The exemption was read from the swap’s `hookData`:

```
1 function _isExemptPlayer(bytes calldata hookData) private view returns (bool) {
2     // BEFORE
3     if (hookData.length < 32) return false;
4     return taxExempt[abi.decode(hookData, (address))];
5 }
```

In v4, `hookData` is supplied by the swap caller. The gacha router honestly tags it with the real player, but a trader swapping *directly* through the `PoolManager` controls `hookData` entirely.

Impact. Any trader could set `hookData = abi.encode(exemptAddress)` and pay **zero hook fee** on every swap — collapsing the protocol’s entire economic engine, since that fee is what funds the floor vault, the genesis dividend, and the self-funding relaunch reserve. No theft of principal, but a total, permanent bypass of fee capture: a protocol-wide value-leak exploitable by anyone, permissionlessly.

Remediation. Honour the exemption only for swaps arriving from a *trusted opener* (the gacha router), whose tag can be believed. A direct swapper’s `sender` is not an opener, so its `hookData` is ignored:

```
1 function _isExemptPlayer(address sender, bytes calldata hookData) private
  view returns (bool) {
2   if (!isOpener[sender]) return false;           // AFTER: only trust the
    router's tag
3   if (hookData.length < 32) return false;
4   return taxExempt[abi.decode(hookData, (address))];
5 }
```

The intended launch buy (deployer → router → pool) still works because the router is a registered opener. **Status: Fixed**, with a regression test asserting the opener gate.

7.2 F-12 — Genesis bonus claimable by non-genesis tokenId (Low)

F-12 · CauldronRegistry · Input Validation

Location: `claimGenesis`.

Description. `claimGenesis(tokenId)` pays a founder’s token bonus per MiFren. The reserve is sized for the genesis tranche (`1..genesisShares`), but the function did not bound `tokenId` to that range. Once iteration 2 mints volume MiFrens (`ids > genesisShares`), those could race the founders for a finite reserve.

Remediation. `if (mifrenTokenId == 0 || mifrenTokenId > genesisShares) revert BadConfig();`
Status: Fixed + tested.

7.3 F-14 — Pluggable policy modules: gas-grief & owner-set surface (Info)

F-14 · CauldronHook · Upgradeability / DoS

Location: `deathChecker`, `surtaxPolicy`, `oddsPolicy`, `curvePolicy` and their delegating call-sites.

Description. To allow the death rule and the surtax / odds / mint curves to be *upgraded without a proxy*, the hook delegates to optional external modules. This adds two surfaces: (i) a module is an external call in the swap hot path, so a pathological module could grief gas; (ii) the modules are owner/registry settable.

Design mitigations (verified sound). The pattern is deliberately defensive: every delegation is (a) wrapped in `try/catch` so a reverting module *always* falls back to the built-in rule and can never brick a swap or relaunch; (b) *clamped* to the same hard caps as the built-ins (`MAX_SNIPE_BPS`, `ODDS_HARD_CAP_BPS`, non-zero curve cost), so a module cannot exceed the surtax ceiling, guarantee a gacha win, or set a zero-cost mint; (c) a pure `view` (staticcall), so it cannot reenter or mutate state; and (d) never in a fund-moving path — modules return *numbers* only. The fund flows (fixed supply, 1:1 migration, LP recycling) remain immutable; only policy math is swappable.

Residual. The setters share the F-03 owner-key surface. Recommendation: route `setDeathChecker/setPolicy` through the registry so they inherit the emergency timelock on mainnet, and hold the hook owner in a multisig. **Status: Mitigated** (bounded + fallback by construction; multisig is the mainnet operational step).

7.4 F-15 — LaunchSniper & finalizer gate centralization (Low)

F-15 · MiFrensGenesis / LaunchSniper · Centralization

Location: `MiFrensGenesis.finalizer, finalize`; `LaunchSniper`.

Description. Genesis `finalize()` (which summons iteration 1 and seeds the pool) was permissionless. A bot could ignite first, breaking the team’s *atomic* launch-plus-buy and forcing a separate, MEV-exposed buy. A `finalizer` gate was added: if set, only that address (the `LaunchSniper`) may finalize. `LaunchSniper.launch()` then performs `finalize` + tax-exempt buy + forward-to-airdrop in one transaction.

Assessment. This is an intentional, one-time, *launch-only* centralization that improves user outcomes (no front-run, no dilution). It gates *only* genesis ignition; all subsequent relaunches remain permissionless, and the finalizer cannot alter fund flows — it can only be the one to press the already-fixed “ignite” button. On mainnet the launch tx should be submitted via a private mempool (Flashbots Protect) for the last mile. **Status: By design.**

8 Revision 3 Findings — Reserve-LP Migration Model

The un-migrated and genesis supply is no longer held as a registry ERC-20 *balance* (which reads as a whale bag on explorers). Instead, at each summon/relaunch the full fixed supply is placed into **two** Uniswap v4 positions, both owned by the registry:

- **Active** (full range): sets the launch price + tradeable depth. Gen-1 seeds it at 10%; every relaunch instead *recycles* it to the tokens still in the dead pool’s LP (the supply that never circulated), so the split self-balances — the reserve auto-sizes to exactly the prior circulating supply (the real 1:1 migration demand), less unclaimed genesis.
- **Reserve** (single-sided token, *out of range below* the launch tick): the other ~90%. Since ETH is `currency0`, pool price is *tokens-per-ETH*; a token that appreciates moves the price *down*, so the reserve sits below launch and is pure `token1` until the token appreciates $\approx 69\times$ into the range (`RESERVE_CEILING_OFFSET` = 42400 ticks $\approx \ln 69 / \ln 1.0001$).

Migration (`claimByBurn`, `autoMigrateBatch`) and the genesis bonus (`claimGenesis`) release tokens 1:1 from the current generation’s reserve via `decreaseLiquidity` + `TAKE_PAIR` to the claimant — pure token, *zero ETH, no price movement*. All V4 encoding lives in the external linked library `PoolOps` (`delegatecall`’d, stateless), which also let the registry runtime shrink from 25,117 to 20,062 bytes. *Verified live on Sepolia*: after summon the registry’s loose token balance was 3.7×10^{-15} of the 777M supply (the entire supply is in the two positions), and a live `claimGenesis` released exactly 69,936.99 tokens (77.7M/1111) from the reserve, decrementing its liquidity while leaving it open.

8.1 F-16 — Reserve exhaustion could under-fill a late migration (Low)

`claimFromReserve` clamps the removed liquidity to what the position holds, so if the reserve were ever short it would return *less* than the requested 1:1 amount after the caller’s old tokens are already burned. In practice the reserve is ~90% of a freshly-minted full supply each generation

while the claimable amount (circulating prior-gen supply + outstanding genesis) is bounded well below that, so exhaustion is not reachable under normal operation. **Recommendation:** add an explicit **require** that the post-removal token-out $\geq$ requested (revert rather than under-fill), so a burn can never settle for less than 1:1. **Status: Acknowledged.**

8.2 F-17 — $A \geq 69\times$ pump sells the reserve wall for ETH (Low)

If the token trades up $\approx 69\times$, spot enters the reserve range and the pool begins converting reserve `token1` into ETH — depleting the token available for 1:1 migration at the top. This is the deliberate ceiling: the reserve doubles as deep sell-side liquidity only at valuations where a holder is strictly better off, and the captured ETH accrues to the protocol-owned position (recycled at relaunch). Migrators arriving post-pump receive ETH-value rather than 1:1 token. **Status: By design** (the $69\times$ ceiling was chosen explicitly; raising `RESERVE_CEILING_OFFSET` hardens the peg further at the cost of moon-time ETH capture).

8.3 F-18 — Pool0ps external-library delegatecall surface (Info)

`Pool0ps` is a `delegatecall`'d linked library: it executes in the registry's context (`this=registry`, which owns the tokens/positions and is the `msg.sender` seen by the `PositionManager/Permit2`). It is **stateless**, contains no `selfdestruct` or `delegatecall` of its own, and only calls the trusted `PoolManager`, `PositionManager`, `Permit2`, and the in-house token. The link address is fixed at deploy time by the toolchain. All reserve/claim calls occur inside `nonReentrant` registry entrypoints, so a `PositionManager` callback cannot re-enter. **Status: Reviewed — no issue.**

8.4 F-19 — Claims are non-sandwichable (Positive)

Because the reserve is *out of range*, `decreaseLiquidity` returns a deterministic token amount and moves spot price by zero. A migration/genesis claim therefore has no MEV surface — there is no price impact to sandwich and `amount1Min` slippage is moot. This is a strict improvement over a held-pool **transfer** model (which was also MEV-free but exposed the supply as a wallet balance). **Status: Positive property.**

9 Positive Security Properties

The following invariants were examined and found **sound**:

- **Non-inflationary migration.** `claimByBurn` burns the caller's previous-generation balance *before* minting the current token 1:1, so cross-iteration supply is conserved and no token can be claimed twice (dead tokens are frozen against transfer).
- **Airdrop follows the NFT.** `claimGenesis` keys on `genesisClaimed[tokenId]` and gates on live `ownerOf`, so the claim right transfers with secondary sales and can never double-pay.
- **Vault solvency.** `CauldronVault.redeem` follows CEI under a reentrancy guard; `floor = balance / outstanding` is preserved across redemptions (both numerator and denominator drop by one share).
- **Dividend accounting.** `MiFrensDividend` uses an $O(1)$ `MasterChef` accumulator with sub-share **residual** carry; the presale's `finalize()` guarantees the genesis tranche is fully minted before any fee accrues, so no share is unclaimable.
- **Snapshot voting.** `CauldronGovernor` weights votes by `getPastVotes` at the proposal snapshot, defeating transfer-and-revote; one vote per address per proposal.

- **Reentrancy.** All external state-changing entrypoints (`summon`, `relaunch`, `claimByBurn`, `claimGenesis`, `vault redeem/close`, `dividend claim`, `router play/playChurn`) carry `nonReentrant` guards and update state before external calls.
- **Arithmetic.** Seed price uses `FullMath.mulDiv` to avoid the `tokenAmount << 192` overflow; credit accrual saturates rather than reverting a swap.
- **Reduced owner surface.** Registry ownership is handed to the autonomous presale post-deploy, freezing owner-only configuration.

10 Test Coverage & Verification

All fixes were compiled under the project’s `cauldron` Foundry profile (`via_ir`) and exercised by the test suite:

Metric	Value
Test suites	10
Tests passing	66 / 66
New regression tests (both engagements)	12
— <code>CauldronTokenFreeze.t.sol</code> (F-02)	2
— <code>CauldronHookAccess.t.sol</code> (F-01)	3
CauldronRegistry runtime	24,458 / 24,576 B · CauldronHook 16,229 (EIP-170)

11 Conclusion

The Cauldron implements an ambitious, largely trust-minimised design: a token lifecycle that funds and perpetuates itself from its own trading fees, with NFT governance selecting each successor and a fee dividend rewarding the founding guild. The core value-conserving invariants — 1:1 migration, vault solvency, dividend accounting, and snapshot voting — are well constructed and defended by reentrancy guards and CEI ordering.

The two engagements identified two High, two Medium, and several lower-severity issues; the High/Medium fund-relevant ones were remediated with minimal, well-scoped changes and are now regression-tested. The residual risk is concentrated in the two privileged roles (F-03), which are mitigated on-chain by an immutable emergency timelock and, for the hook, by the new write-once registry binding. We recommend the mainnet operational hardening in F-03 (multisig custody of both privileged roles, with a published decommissioning path) as the final step toward the protocol’s stated goal of an “unstoppable, autonomous” machine.

End of report. Findings reflect the codebase state at the time of review; any subsequent change should be re-audited.